CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

NAAC GRADE A+
Accredited University

MINOR PROJECT REPORT

DeployFlow – CI/CD Pipeline with Kubernetes Orchestration

Student Name: Simran **UID:** 24MCC20089
Branch: MCA(CCD) **Section/Group:** 24MCC102/A
Semester: 4th **Date of Performance:** 2-4-2026
Subject: Kubernetes for DevOps **Subject Code:** 24CAH-783

ABSTRACT

This project presents an automated CI/CD pipeline integrated with Kubernetes for efficient application deployment and management. The system uses a web-based DevOps dashboard (DeployFlow) that simulates deployment monitoring and integrates with Jenkins for automation. GitHub is used for version control, Jenkins for continuous integration and deployment, Docker for containerization, and Kubernetes for orchestration.

Whenever code is updated and pushed to GitHub, Jenkins automatically triggers the pipeline, builds a Docker image, and deploys it. Kubernetes manages the deployed application by ensuring availability, scalability, and fault tolerance through features like self-healing and load balancing. The system demonstrates a modern DevOps workflow that reduces manual effort and ensures reliable and scalable deployments.

PROBLEM STATEMENT

In traditional development workflows, deploying applications manually is time-consuming and prone to errors. Developers must repeatedly build, package, and deploy applications, which reduces productivity and increases the risk of inconsistencies. Additionally, managing application availability and scaling manually is inefficient. If an application crashes, it must be restarted manually, and handling increased traffic becomes difficult.

This project addresses these issues by implementing an automated CI/CD pipeline with Kubernetes, enabling automatic deployment, scaling, and recovery of applications without manual intervention.

OBJECTIVE

The main objectives of this project are:

- To automate application deployment using Jenkins CI/CD pipeline
- To containerize the application using Docker
- To manage and orchestrate containers using Kubernetes
- To demonstrate self-healing and scaling capabilities
- To reduce manual deployment effort and improve efficiency

TOOLS & TECHNOLOGIES USED

- **Version Control:** Git, GitHub
- **CI/CD Tool:** Jenkins
- **Containerization:** Docker
- **Orchestration:** Kubernetes (Minikube)
- **Backend:** Node.js (Express)
- **Frontend:** HTML, CSS, JavaScript
- **Database:** MongoDB
- **Operating System:** Windows

SYSTEM DESIGN

The system consists of the following components:

1. **GitHub Repository**
   Stores application code and triggers CI/CD pipeline on code push.
2. **Jenkins Server**
   Automates build and deployment process using pipeline.
3. **Docker**
   Packages application into containers for consistent execution.
4. **Kubernetes Cluster (Minikube)**
   Manages container deployment, scaling, and recovery.
5. **DeployFlow Dashboard**
   Displays deployment status, logs, and performance metrics.

SYSTEM WORKFLOW

Developer → GitHub → Jenkins → Docker → Kubernetes → Running Application

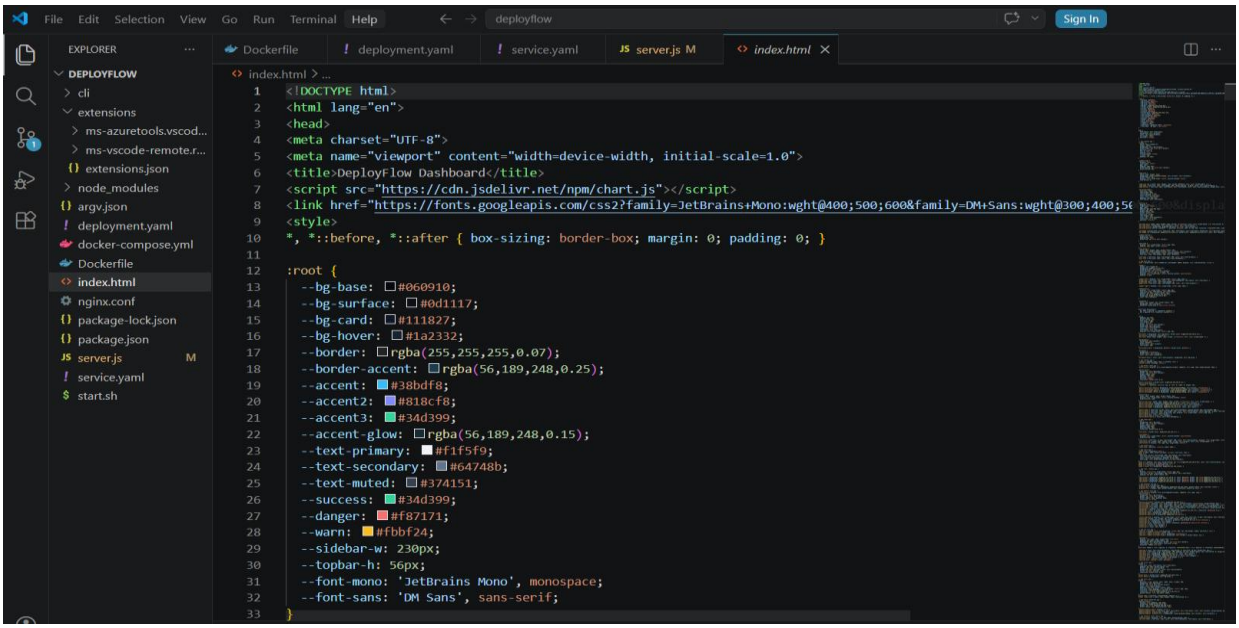
WORKFLOW STEPS:

- Developer pushes code to GitHub
- Jenkins pipeline is triggered
- Docker image is built
- Kubernetes deploys the container
- Application runs as pods
- Dashboard displays deployment status

IMPLEMENTATION

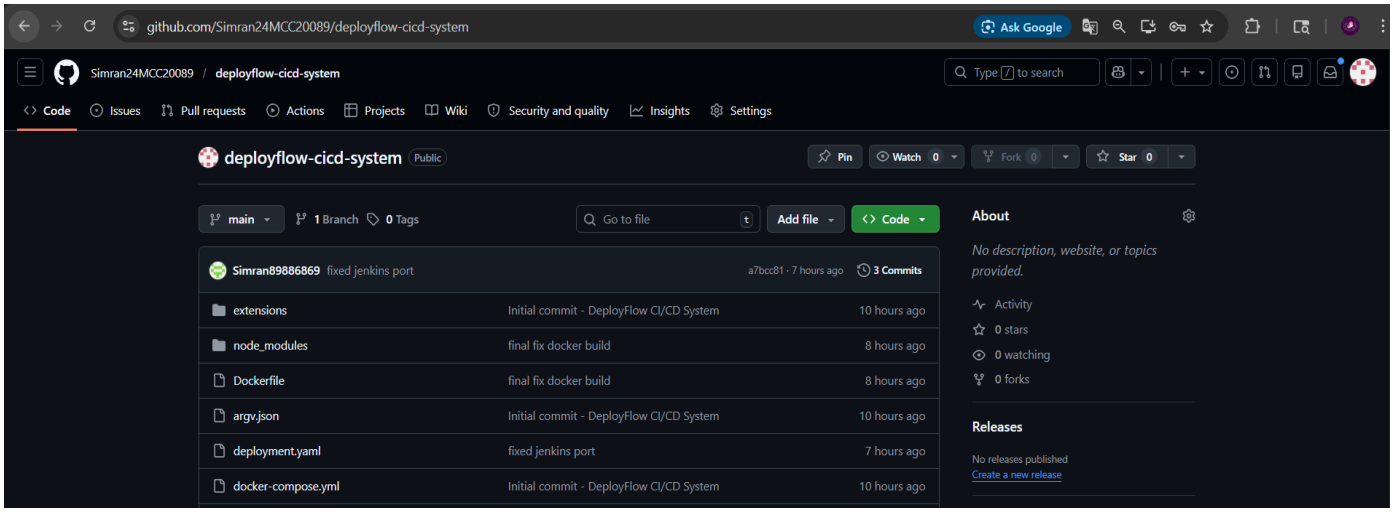
Step 1: Application Development

A web-based DevOps dashboard named *DeployFlow* was developed using HTML, CSS, JavaScript, and Node.js (Express). The application displays deployment statistics, logs, and system status. It also includes interactive features such as a deploy button to simulate deployment activity.



Step 2: Version Control using GitHub

The project was initialized using Git and pushed to a GitHub repository. This repository serves as the central codebase and is used by Jenkins to fetch the latest code during pipeline execution.



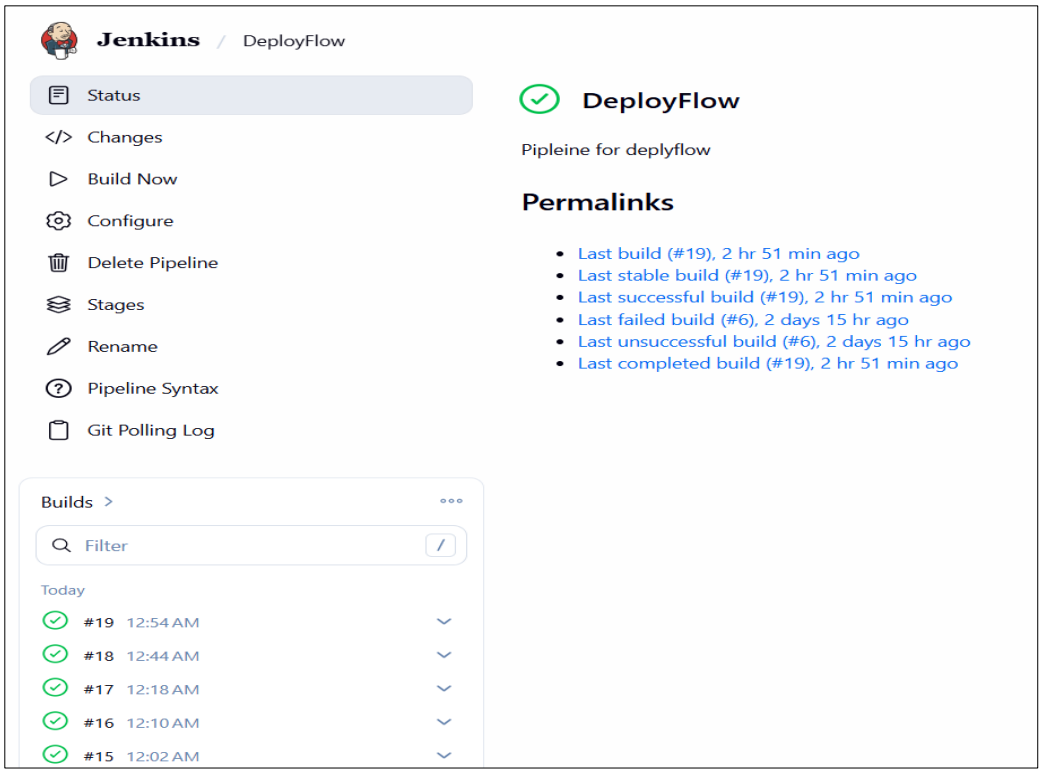
Step 3: Docker Containerization

A Dockerfile was created to package the application into a container. The image was built and tested locally to ensure that the application runs properly in a consistent environment.

```
PS C:\Users\Simran\deployflow> docker-compose up --build
time="2026-04-13T11:41:39+05:30" level=warning msg="C:\\Users\\Simran\\deployflow\\docker-compose.yml:
emove it to avoid potential confusion"
[+] Building 8.7s (13/13) FINISHED
```

Step 4: Jenkins CI/CD Pipeline Setup

Jenkins was configured as the CI/CD tool and a pipeline was created. The pipeline automates the process of fetching code from GitHub, building the Docker image, and deploying the application. The pipeline can be triggered manually or automatically.



Step 5: Kubernetes Setup (Minikube)

Minikube was used to set up a local Kubernetes cluster. This provides an environment to deploy and manage containerized applications.

```
PS C:\Users\Simran\deployflow> minikube start
😄  minikube v1.38.0 on Microsoft Windows 11 Home Single Language 25H2
✨  Using the docker driver based on existing profile
🎉  minikube 1.38.1 is available! Download it: https://github.com/kubernetes/minikube/releases/tag/v1.38.1
💡  To disable this notice, run: 'minikube config set WantUpdateNotification false'
```

Step 6: Deployment in Kubernetes

A deployment configuration file (deployment.yaml) was created to define how the application should run inside Kubernetes. It specifies container details and replica count. The deployment was applied using kubectl commands, which created running pods.

```
PS C:\Users\Simran\deployflow> kubectl apply -f deployment.yaml
deployment.apps/deployflow-deployment created
```

Step 7: Service Configuration

A service configuration file (service.yaml) was used to expose the application externally using NodePort. This allows users to access the application through a browser.

```
PS C:\Users\Simran\deployflow> kubectl apply -f service.yaml
service/deployflow-service created
```

Step 8: Demonstration of Kubernetes Features

Kubernetes features such as self-healing and scaling were demonstrated. When a pod was deleted, Kubernetes automatically recreated it. The application was also scaled to multiple replicas to handle increased load.

DEMO 1: SELF-HEALING (Pod Auto-Recovery)

1: Show running pods

kubectl get pods

```
PS C:\Users\Simran\deployflow> kubectl get pods
NAME                                  READY   STATUS             RESTARTS      AGE
cicd-app-57bd48b6b9-5fjnr             0/1     ImagePullBackOff   0             6d14h
deployflow-deployment-67d85b4646-v2twf   1/1     Running            0             7m47s
```

“Currently, one pod of my application is running.”

2: Delete the pod

kubectl delete pod deployflow-deployment-67d85b4646-bh2lb

```
PS C:\Users\Simran\deployflow> kubectl delete pod deployflow-deployment-67d85b4646-v2twf
pod "deployflow-deployment-67d85b4646-v2twf" deleted
```

3: Watch live recovery

kubectl get pods -w

```
PS C:\Users\Simran\deployflow> kubectl get pods -w
NAME                                  READY   STATUS             RESTARTS   AGE
cicd-app-57bd48b6b9-5fjnr             0/1     ImagePullBackOff   0          13d
deployflow-deployment-67d85b4646-bh2lb   1/1     Running            0          2m7s
```

- A new pod name will appear.

"Even after deleting the pod manually, Kubernetes automatically recreates it. This is called self-healing, which ensures application availability."

DEMO 2: SCALING (Handling Load)

Step 1: Scale to 3 replicas

kubectl scale deployment deployflow-deployment --replicas=3

```
PS C:\Users\Simran\deployflow> kubectl scale deployment deployflow-deployment --replicas=3
deployment.apps/deployflow-deployment scaled
```

Step 2: Verify scaling

kubectl get pods

```
PS C:\Users\Simran\deployflow> kubectl get pods
NAME                                  READY   STATUS             RESTARTS   AGE
cicd-app-57bd48b6b9-5fjnr             0/1     ImagePullBackOff   0          13d
deployflow-deployment-67d85b4646-bh2lb   1/1     Running            0          4m6s
deployflow-deployment-67d85b4646-p8c9h   1/1     Running            0          7s
deployflow-deployment-67d85b4646-rbxm5   1/1     Running            0          7s
```

- 3 pods running

Step 9: Final Output

The application was successfully deployed and accessed through the browser. All components including Jenkins, Docker, and Kubernetes worked together to provide a complete automated deployment system.

```
PS C:\Users\Simran\deployflow> minikube service deployflow-service
|-----------|--------------------|-------------|-------------------------|
| NAMESPACE |        NAME        | TARGET PORT |           URL           |
|-----------|--------------------|-------------|-------------------------|
| default   | deployflow-service | 80          | http://192.168.49.2:30008 |
|-----------|--------------------|-------------|-------------------------|
🔗  Starting tunnel for service deployflow-service.
|-----------|--------------------|-------------|------------------------|
| NAMESPACE |        NAME        | TARGET PORT |          URL           |
|-----------|--------------------|-------------|------------------------|
| default   | deployflow-service |             | http://127.0.0.1:57165 |
|-----------|--------------------|-------------|------------------------|
🎉  Opening service default/deployflow-service in default browser...
❗  Because you are using a Docker driver on windows, the terminal needs to be open to run it.
```

RESULTS & OBSERVATIONS

The project successfully demonstrated an automated CI/CD pipeline integrated with Kubernetes. The application was deployed successfully, and the dashboard displayed deployment information accurately. Jenkins was able to automate the build and deployment process without manual intervention.Kubernetes effectively managed the application by ensuring that it remained available at all times. When a pod was deleted, it was automatically recreated, demonstrating the self-healing capability. Scaling the application increased the number of running pods, which showed the scalability feature.Overall, the system performed efficiently and reduced the need for manual deployment. The integration of multiple DevOps tools resulted in a robust and reliable deployment system.

CONCLUSION

Key Achievements:

- Automated CI/CD pipeline using Jenkins
- Containerized application using Docker
- Managed deployment using Kubernetes
- Demonstrated self-healing and scaling

Future Improvements:

- Deploy application on cloud(AWS, GCP, Azure)
- Add real-time monitoring tools(Prometheus, Grafana)
- Implement security scanning in pipeline

Challenges Faced:

- Docker networking issues while connecting Jenkins
- Kubernetes configuration errors (NodePort conflicts)
- Webhook limitations on localhost